NRLC-04: Alert & Workflow Migration

> **Series:** NRLC | **Notebook:** 4 of 9 | **Created:** April 2026 | **Last Updated:** 04/15/2026

Overview

Alerts are the highest-stakes migration artifact — they wake people up. This deep dive covers the structural mismatch between NR alert policies and Gen3 Dynatrace, where alert routing lives in **Workflows** (not the Gen2 Alerting Profile object). It covers how NRQL conditions become Davis anomaly detectors, why APM conditions still merit review even though Phase 17 now auto-converts them, the notification mapping (Workflow tasks instead of standalone Notification Channels), and the dual-alert window pattern that protects on-call during cutover.

Phase 17 alert coverage (post-2026-04-15): the engine now auto-converts **NRQL conditions**, **NRQL baseline / outlier conditions** (`baseline_alert_transformer`), **Infrastructure / Synthetic / Browser / Mobile / External-service / Multi-location-synthetic conditions** (`non_nrql_alert_transformer`), **mute rules + scheduled + recurring maintenance windows** (`maintenance_window_transformer`), **lookup-table WHERE-IN subqueries** (`lookup_table_transformer` → Resource Store JSONL + DQL `lookup` subquery), and **deployment markers / change events** (`change_tracking_transformer` → DT events API `CUSTOM_DEPLOYMENT` / `CUSTOM_CONFIGURATION`). **Phase 18** adds `aiops_transformer` for NR AI Workflows / destinations / enrichments (with the NR↔DT "Workflow" name-collision callout). See COVERAGE-MATRIX.md §7 for the full alert-family row-set.

Table of Contents

4. NRQL Condition → Metric Event
5. APM Condition → Davis Adaptive Baseline
6. Notification Channel Mapping
7. Dual-Alert Window Pattern
8. Mute Rules & Maintenance Windows

Prerequisites

Requirement	Details
Audience	On-call leads, SRE, platform engineers wiring alert paths
Standalone	This notebook is self-contained for alert + workflow migration. No required prerequisite reading.
Optional depth	NRLC-02 (full NRQL→DQL compiler), NRLC-08 (validation), NRLC-09 (toolchain)
High-stakes warning	Alert migration affects on-call. Plan a dual-alert window of 1–2 weeks before NR alerts are silenced.

Embedded Translation Context — Alert Condition Query Patterns

Each NRQL alert condition contains a query that becomes a DQL Metric Event. The patterns below cover the vast majority of alert conditions — enough to migrate alerts without consulting the full NRLC-02 compiler reference.

Threshold on simple count

```
-- NRQL
SELECT count(*) FROM TransactionError WHERE appName = 'checkout'
-- threshold: above 10 for at least 5 minutes
```

```
-- DQL (Metric Event query)
fetch spans, from:-5m
| filter service.name == "checkout" AND isNotNull(error)
| summarize count = count()
-- threshold: count &gt; 10
```

Percentile threshold

```
-- NRQL
SELECT percentile(duration, 95) FROM Transaction WHERE appName = 'api'
-- threshold: above 1.0 for at least 5 minutes
```

```
-- DQL
fetch spans, from:-5m
| filter service.name == "api"
| summarize p95 = percentile(duration, 95)
-- threshold: p95 &gt; duration("1s")
```

Error rate threshold

```
-- NRQL
SELECT percentage(count(*), WHERE error) FROM Transaction
-- threshold: above 5 (= 5%) for 5 minutes
```

```
-- DQL
fetch spans, from:-5m
| summarize error_pct = 100.0 * countIf(isNotNull(error)) / count()
-- threshold: error_pct &gt; 5
```

Translation Confidence for Alerts

Condition pattern	Confidence	Notes
Static threshold on count/avg/sum	HIGH	Direct
Percentile threshold	HIGH	`percentile(field, N)` syntax
Percentage threshold	HIGH	`100.0 * countIf(x) / count()`

Condition pattern	Confidence	Notes
Baseline (NR adaptive)	MEDIUM → LOW	Prefer Davis adaptive baselines instead of porting
APM condition (built-in error rate, etc.)	LOW	Manual: configure Davis baseline

1. NR Alert Model

NR's alert model is a 3-tier structure:

```
Alert Policy
  ├─ Alert Condition (NRQL, APM, Infrastructure, Synthetic, ...)
  │   ├─ Threshold (warning, critical)
  │   └─ Duration / Aggregation
  └─ Notification Channel (Email, Slack, PD, Webhook, ...)
```

Conditions are typed:
- **NRQL Condition** — query + threshold; most flexible, most common
- **APM Condition** — prebuilt detection (error rate, response time, throughput)
- **Infrastructure Condition** — host/process metrics with thresholds
- **Synthetic Condition** — monitor-result-based
- **External Service Condition** — cross-service detection

The migration target depends on the source condition type.

2. DT Alert Model

Dynatrace Gen3 splits NR's monolithic alert into two layers, with **Workflows** replacing the Gen2 Alerting Profile + Notification Channel pattern:

```
Metric Event (the detection)
  │ schema: builtin:problem.metric.events
  │ query (DQL or metric expression)
  │ threshold + operator
  │ duration (n out of m samples)
  └──── raises a Davis Problem (event)
            │
            └── Workflow (the routing + action layer)
                  │ trigger: Davis problem event
                  │ filter: severity, entity attributes, tags, bucket
                  └── Tasks (run in sequence or parallel):
                        │ send_slack / send_email / send_pagerduty
                        │ send_webhook / run_javascript
                        └── trigger downstream automation
```

The decoupling is intentional: a single Davis problem can match multiple Workflows (different teams react differently). Notification *delivery* lives inside Workflow tasks rather than as a separate object — this is the Gen3 model. The legacy Alerting Profile + Notification Channel pattern still exists for backward compatibility but is **not** the recommended Gen3 path.

3. Policy → Workflow

An NR Alert Policy + its Notification Channels maps to a Gen3 **Workflow** in Dynatrace.

NR Concept	Gen3 Workflow Equivalent
`name`	Workflow name
`incidentPreference` (PER_POLICY / PER_CONDITION / PER_CONDITION_AND_TARGET)	Workflow trigger filters + grouping settings
Channel binding (channel.id list)	Workflow tasks (one task per delivery target)
Channel-level filters	Filter step on the Davis problem trigger

The `AlertTransformer` creates one Workflow per NR policy:

- **Trigger:** Davis problem event with filter conditions matching the policy's intended scope (severity, affected entities by attribute, bucket scope)

- **Tasks:** one per original notification channel (e.g., a policy with Email + Slack + PagerDuty becomes one Workflow with three tasks)
- **Conditional branches:** for `PER_CONDITION` policies, separate trigger filters per condition; for `PER_POLICY` grouping, a single trigger covers all conditions

Delivery targets are deduplicated at the **task** level, not at a separate Notification-object level. Two Workflows that both send to the same Slack channel each reference the channel directly in their respective `send_slack` task.

4. NRQL Condition → Metric Event

Each NR NRQL condition becomes a DT Metric Event. The translation pipeline:

```
NR NRQL Condition
      ↓
  [Compile NRQL → DQL via nrql-engine]
      ↓
  [Map threshold + operator]
      ↓
  [Map duration (e.g., 5 minutes) → evaluation window]
      ↓
  Emit DT Metric Event
```

Threshold mapping:

NR	DT
`thresholdType: STATIC` + `operator: ABOVE`	`monitoringStrategy: STATIC_THRESHOLD` + `alertCondition: ABOVE`
`thresholdType: BASELINE`	DT auto-adaptive baseline (recommended; configures Davis)
`terms.priority: CRITICAL`	`eventType: ERROR`
`terms.priority: WARNING`	`eventType: INFO` (with severity tag)

Duration mapping:

NR	DT
`5 minutes`	`samples: 5, violatingSamples: 5` (1-min granularity)

NR	DT
`at least once in 5 minutes`	`samples: 5, violatingSamples: 1`
`for at least 5 minutes`	`samples: 5, violatingSamples: 5`

Confidence consideration: the NRQL must compile to HIGH-confidence DQL. MEDIUM/LOW translations should be hand-validated before becoming production alerts.

5. APM / Non-NRQL Condition → Davis Adaptive Baseline

NR's APM conditions ("alert when error rate above 5% for 10 minutes") use a built-in detection model. DT solves the same problem with **Davis adaptive baselines** — but the configuration is fundamentally different:

- NR sets a **fixed threshold** with a sensitivity dial.
- DT learns the **expected baseline** for each entity and alerts on statistically significant deviation.

Phase 17 auto-conversion: `non_nrql_alert_transformer` now handles Infrastructure / Synthetic / Browser / Mobile / External-service / Multi-location-synthetic conditions, and `baseline_alert_transformer` handles NR baseline/outlier NRQL conditions. All emit `builtin:davis.anomaly-detectors` configuration — no manual porting required. Review is still recommended for threshold tuning.

NR Condition	Engine Output	Recommended DT Refinement
Error rate above N% (APM)	Davis error-rate anomaly detector	Raise/lower sensitivity based on baseline width
Response time above N ms	Davis response-time adaptive detector	Consider pairing with SLO for burn-rate alerts
Throughput drop	Davis throughput adaptive baseline	—

NR Condition	Engine Output	Recommended DT Refinement
Apdex below N	Custom DT Apdex calculation + DQL anomaly detector (DT has no native Apdex)	Phase 19 apdex-bucket uplift raises compiler confidence to HIGH for bucketed queries
NR baseline condition	Davis adaptive baseline (direct)	—
NR outlier condition	Davis outlier detector	—
Multi-location synthetic	Davis detector with location-count expression	Adjust minimum-failed-locations threshold

Review pattern: the engine imports the signal shape (which metric, which entity scope, which direction), but Davis's adaptive nature means DT will fire on different events than NR did. The dual-alert window (§7) catches this — expect DT volume to deviate from NR volume in both directions, and tune from there.

6. Notification → Workflow Task Mapping

In Gen3, notifications are not standalone objects — they are **tasks inside a Workflow**. Each NR notification channel becomes a task in the migrated Workflow.

NR Channel	Workflow Task	Migration Notes
Email	`send_email` task	Email addresses copy directly into task config
Slack	`send_slack` task	Webhook URL stored in task; **must be reissued** (NR's URL won't work)
PagerDuty	`send_pagerduty` task	Service Integration Key stored in task; **must be regenerated**
OpsGenie	`send_opsgenie` task	API key in task config; **reissue required**
ServiceNow	`send_servicenow` task or `http_request`	Connector configuration replicated
Webhook	`http_request` task	URL + headers + payload template migrate; **secrets do not**

NR Channel	Workflow Task	Migration Notes
User (channel)	`send_email` task	Resolved to user's email
xMatters	`http_request` task	xMatters webhook URL

Critical: every task that uses a secret (token, webhook URL, API key) **must have its secret rotated and re-entered** in the Workflow task config or in the Dynatrace credentials vault. The transformer can't migrate secrets — it emits placeholder values that must be filled in before the Workflow is enabled.

Workflow task ordering: tasks within a Workflow run in declared order by default (or in parallel if explicitly configured). Migration places critical tasks (PagerDuty) first so they fire even if a later task (e.g., a downstream automation script) fails.

7. Dual-Alert Window Pattern

Never silence NR alerts on the same day DT alerts go live. Run both for 1–2 weeks.

The Pattern

1. Migrate alerts to DT, but route notifications to a **silent test channel** (e.g., a Slack channel only the SRE team sees).
2. Run for 7–14 days.
3. Compare alert volume: DT should match NR within ±10%. If higher, DT is over-alerting (tune thresholds). If lower, DT may be missing detections (review condition logic).
4. Compare alert content: same problem detected at roughly the same time on both platforms?
5. Once dual-alert volume aligns, swap DT to the production channel and silence NR for that policy.
6. Migrate next policy. Don't batch — cutover one policy at a time.

Volume Comparison Query (DQL)

```
fetch events, from:-7d
| filter event.kind == "DAVIS_PROBLEM"
| summarize problems = count(), by:{event.name, event.category}
```

Compare against NR's `Issues` count for the same period. Investigate any condition with > 25% delta.

8. Mute Rules & Maintenance Windows

Phase 17 auto-conversion via `maintenance_window_transformer`:

NR Concept	DT Equivalent	Engine Output
Mute Rule (NRQL-based)	`builtin:deployment.maintenance` with filter expression	Direct translation
Scheduled Maintenance Window	`builtin:deployment.maintenance` (scheduled)	Direct
Recurring Window	Recurrence schedule	Direct

Mute rules that filter on metric values (e.g., "mute when load < 10") translate — the transformer converts them to `builtin:davis.anomaly-detectors` with an embedded filter expression rather than a separate maintenance window, because DT maintenance windows scope by entity not by signal value.

Summary

Alert migration is the highest-risk phase: errors here wake people up. Post-Phase-17, the pattern is:

1. Auto-convert NRQL + APM + non-NRQL + baseline + outlier conditions (engine handles all via Phase 17 transformers)
2. Review imported detectors — Davis adaptive behavior differs from NR fixed thresholds
3. Recreate notification channel secrets manually (never auto-migrated)

4. Auto-convert mute rules + maintenance windows (Phase 17)
5. Run dual-alert for 1–2 weeks before silencing NR — use `--canary <pct>` for production tenants
6. Cut over one policy at a time

Continue to **NRLC-05 Synthetic Monitor Migration** for monitor-type-by-monitor-type migration.

Tooling for Alert-Only Migration

End-to-end commands for migrating only alerts (Metric Events + Workflows). Notification channels become Workflow tasks; secrets must be re-entered.

```
# 1. Inventory NR alert policies + conditions + notification channels
python3 migrate.py --export-only --components alerts,notifications --output
./alerts-export

# 2. Translate condition queries; emit Metric Events + Workflows
python3 migrate.py --transform-only --components alerts,notifications --
report

# 3. Diff (avoid recreating alerts that were partially migrated previously)
python3 migrate.py --diff --components alerts,notifications

# 4. Import to silent test channel first (route to staging Workflow target)
python3 migrate.py --import-only --components alerts,notifications

# 5. Run dual-alert comparison for 1–2 weeks
python3 migrate.py compare-alerts --window 7d

# 6. Promote to production channels (re-enter secrets in Workflow tasks)
# 7. Silence NR alerts for the migrated policies
```

Workflow validation:

```
# Confirm each migrated Workflow triggers correctly
python3 migrate.py validate-workflows --components alerts
```

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source Dynatrace-NewRelic, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.